RAG-Driven Customer Support Assistant System.
Deliverable 3- Technical Document

ADVANCED_GENAI_INTERNSHIP – FEBRUARY 2026

NAMRATA DIGHE - IN226091702

1. Introduction

What is RAG (Retrieval-Augmented Generation)?

Retrieval-Augmented Generation (RAG) is an AI architecture that combines:

- **Information Retrieval (IR)** → fetch relevant data
- **Language Model (LLM)** → generate answers

Instead of relying only on pre-trained knowledge, RAG uses **external data (PDFs, documents)** to produce accurate responses.

Why it is Needed

Traditional LLMs:

- May generate **hallucinated answers**
- Lack domain-specific knowledge

RAG solves this by:

- Grounding responses in real documents
- Improving accuracy and reliability

Use Case Overview

This system is designed for:

- Document-based Question Answering
- Customer support knowledge systems
- Educational content retrieval

Example:

User asks a question → system retrieves relevant PDF content → generates accurate answer

---

2. System Architecture Explanation

The system follows a dual-phase architecture:

Phase 1: Offline Processing

- PDF → Text extraction
- Chunking → Splitting into smaller parts
- Embedding → Convert to vectors
- Storage → Save in ChromaDB

Phase 2: Online Query Processing

- User query input
- Retrieval of relevant chunks

- LLM generates answer
- Output returned to user

Component Interactions

- Document Loader → Chunking → Embedding → Vector DB
- User Query → Retriever → LLM → Response
- LangGraph → Controls workflow and routing
- HITL → Handles low-confidence cases

---

3. Design Decisions

Chunk Size Choice

- chunk_size = 1000
- chunk_overlap = 100

Reason:

- Large enough for context
- Small enough for fast retrieval

Embedding Strategy

- Use semantic embeddings
- Converts text into vector space
- Enables similarity-based retrieval

Retrieval Approach

- Top-K similarity search
- Filters most relevant chunks
- Reduces noise

Prompt Design Logic

- Combine:
  - User query
  - Retrieved context
- Ensure:
  - Answer is based only on provided context
- Prevent hallucination

---

4. Workflow Explanation

LangGraph Usage

- LangGraph is used for:
- Managing workflow
- Handling state
- Enabling conditional routing

Node Responsibilities

Processing Node:

- Retrieve documents
- Generate answer
- Evaluate confidence

Output Node:

- Return final answer
- Or escalate

State Transitions

State object tracks:

- Query
- Retrieved documents
- Generated answer
- Confidence score

Flow:

Query → Retrieval → Generation → Decision → Output

---

5. Conditional Logic

Intent Detection

- Detect if query is:
  - Answerable from documents
  - Out-of-scope

Routing Decisions

- If confidence ≥ 0.8 → Answer
- If confidence < 0.8 → Escalate

---

6. HITL Implementation

Role of Human Intervention

- Handles:
  - Low-confidence answers
  - Missing context
  - Complex queries

Benefits

- Improves accuracy
- Prevents wrong answers
- Builds trust

Limitations

- Slower response time
- Requires human effort

---

7. Challenges & Trade-offs

Retrieval Accuracy vs Speed

- High accuracy → slower retrieval
- Fast retrieval → may reduce relevance

Chunk Size vs Context Quality

- Large chunks → better context
- Small chunks → faster but less context

Cost vs Performance

- High-quality LLM → expensive
- Smaller models → cheaper but less accurate

---

8. Testing Strategy

Testing Approach

- Unit testing for modules
- End-to-end testing for pipeline
- Edge case testing

Sample Queries

- "What is X topic in document?"
- "Explain Y concept"
- Out-of-scope query (to test HITL)

---

9. Future Enhancements

- Multi-Document Support: Handle multiple PDFs
- Feedback Loop: Learn from user corrections
- Memory Integration: Maintain conversation history
- Deployment: Web app using Streamlit / FastAPI

Conclusion

This system provides a **robust RAG-based architecture** that ensures:

- Accurate answers
- Reduced hallucination
- Scalable and modular design

It combines **retrieval, LLM, workflow orchestration, and HITL** to deliver a reliable AI solution.